“Program Execution & Resource Monitoring System”

Understanding how programs execute internally and how system resources are consumed during execution is a core objective of the Elements of Computing (EOC) course. While students learn about instruction execution, control flow, and memory usage theoretically, the internal behavior of running programs often remains abstract and difficult to visualize. This gap between theory and practical understanding motivates the need for tools that can illustrate how programs behave during execution in a simple and controlled manner.

This project presents a Program Execution & Resource Monitoring System that simulates the execution of multiple programs and analyzes their execution flow and resource usage at a conceptual level. The system is designed as an educational execution environment rather than a real operating system or hardware-level simulator. Each program is treated as an independent task and is executed step by step using a controlled execution engine. During execution, the system tracks key execution parameters such as instruction count, execution order, execution time, and memory usage. These parameters provide insight into how different programs consume computational resources and how execution behavior varies based on program structure.

The execution engine models logical instruction execution by breaking program behavior into discrete execution steps. An instruction tracking module records the sequence and frequency of executed steps, enabling analysis of loops, conditional branches, and function calls. Alongside this, a resource monitoring module tracks conceptual CPU usage by measuring execution time and monitors memory usage by recording allocation, deallocation, and peak memory consumption during program execution. The monitoring process does not rely on kernel-level system calls or real hardware measurements; instead, it focuses on illustrating execution and resource utilization principles in a simplified and platform-independent manner.

To improve clarity and understanding, the collected execution and resource data is visualized using execution timelines, comparison tables, and resource usage graphs. These visualizations help users interpret execution behavior, compare multiple programs, and understand the relationship between program logic and resource consumption. By observing how different programs execute internally, users gain a clearer understanding of execution flow, instruction-level behavior, and memory usage patterns.

The proposed system emphasizes conceptual accuracy and educational value rather than performance optimization or hardware precision. It partially applies EOC concepts such as instruction execution, control flow, and memory abstraction while remaining accessible and easy to understand. This project effectively bridges the gap between

high-level programming constructs and low-level execution concepts, making abstract computing principles more concrete and intuitive for learners.

Thank you Mam,

Group 9

Chunduru Venkata Abhiram , Dhiren Vasudeva

Annareddy Pardhav Reddy , Poovya S